

```

import pandas as pd
!pip install prefixspan
from prefixspan import PrefixSpan

# -----
# Load Dataset
# -----
df = pd.read_csv('/content/House Price Prediction Dataset.csv')

# -----
# Step 1: Create Temporal Sequences
# -----
# Convert continuous data into categories (important!)

def categorize_area(x):
    if x < 800:
        return "Small"
    elif x < 1200:
        return "Medium"
    else:
        return "Large"

def categorize_price(x):
    if x < df['Price'].median():
        return "LowPrice"
    else:
        return "HighPrice"

df['Area_Category'] = df['Area'].apply(categorize_area)
df['Price_Category'] = df['Price'].apply(categorize_price)

# -----
# Step 2: Create Sequences
# -----
# Each row becomes a sequence (temporal order assumed)

sequences = df[['Area_Category', 'Price_Category']].values.tolist()

# -----
# Step 3: Encode items to numbers
# -----
unique_items = list(set(item for seq in sequences for item in seq))
item_to_int = {item: i for i, item in enumerate(unique_items)}
int_to_item = {i: item for item, i in item_to_int.items()}

encoded_sequences = [[item_to_int[item] for item in seq] for seq in sequences]

# -----
# Step 4: Apply PrefixSpan
# -----
ps = PrefixSpan(encoded_sequences)

min_support = 3
frequent_sequences = ps.frequent(min_support)

# -----
# Step 5: Decode Results
# -----
print("Frequent Temporal Patterns:\n")

for support, seq in frequent_sequences:
    decoded_seq = [int_to_item[i] for i in seq]
    print(f"Sequence: {decoded_seq}, Support: {support}")

```

```

Collecting prefixspan
  Downloading prefixspan-0.5.2.tar.gz (10 kB)
  Preparing metadata (setup.py) ... done
Collecting docopt>=0.6.2 (from prefixspan)
  Downloading docopt-0.6.2.tar.gz (25 kB)
  Preparing metadata (setup.py) ... done
Collecting extratools>=0.8.1 (from prefixspan)
  Downloading extratools-0.8.2.1.tar.gz (25 kB)
  Preparing metadata (setup.py) ... done
Collecting sortedcontainers>=1.5.10 (from extratools>=0.8.1->prefixspan)
  Downloading sortedcontainers-2.4.0-py2.py3-none-any.whl.metadata (10 kB)
Requirement already satisfied: toolz>=0.9.0 in /usr/local/lib/python3.12/dist-packages (from extratools>=0.8.1->prefixspan)
Downloading sortedcontainers-2.4.0-py2.py3-none-any.whl (29 kB)
Building wheels for collected packages: prefixspan, docopt, extratools
  Building wheel for prefixspan (setup.py) ... done
  Created wheel for prefixspan: filename=prefixspan-0.5.2-py3-none-any.whl size=11215 sha256=cfe3b5c97256971b6f50c26e1f01164
  Stored in directory: /root/.cache/pip/wheels/72/36/65/f9e2bbe4998ab4cdf6e7a015f38bdbb6925301dcf2cce34a88
  Building wheel for docopt (setup.py) ... done

```

```
Created wheel for docopt: filename=docopt-0.6.2-py2.py3-none-any.whl size=13706 sha256=ffb4234e706c3963dae94359f7631047119
Stored in directory: /root/.cache/pip/wheels/1a/bf/a1/4cee4f7678c68c5875ca89eaccf460593539805c3906722228
Building wheel for extratools (setup.py) ... done
Created wheel for extratools: filename=extratools-0.8.2.1-py3-none-any.whl size=28865 sha256=673602d83f7e90715be4e95309e10
Stored in directory: /root/.cache/pip/wheels/0f/ab/1f/a5cf2e0513d68e4ff783d169add0c94235bddf2bb8f3dcd581
Successfully built prefixspan docopt extratools
Installing collected packages: sortedcontainers, docopt, extratools, prefixspan
Successfully installed docopt-0.6.2 extratools-0.8.2.1 prefixspan-0.5.2 sortedcontainers-2.4.0
Frequent Temporal Patterns:
```

```
Sequence: ['Large'], Support: 1713
Sequence: ['Large', 'LowPrice'], Support: 866
Sequence: ['Large', 'HighPrice'], Support: 847
Sequence: ['LowPrice'], Support: 1000
Sequence: ['Medium'], Support: 163
Sequence: ['Medium', 'LowPrice'], Support: 80
Sequence: ['Medium', 'HighPrice'], Support: 83
Sequence: ['HighPrice'], Support: 1000
Sequence: ['Small'], Support: 124
Sequence: ['Small', 'HighPrice'], Support: 70
Sequence: ['Small', 'LowPrice'], Support: 54
```

```
import matplotlib.pyplot as plt

lengths = [len(seq) for _, seq in frequent_sequences]

plt.hist(lengths)
plt.xlabel("Sequence Length")
plt.ylabel("Frequency")
plt.title("Temporal Pattern Length Distribution")
plt.show()
```

